

FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)



Course Code: COMP451

Session: SP26

Assignment Number: 1

Title of Lab/Assignment: Lexical Analyzer

Group Member/s Names and Roll Number/s: Musfira Sehar, 280048443

Table of Contents

1. Introduction
2. Objective
3. Background
4. Tokens Supported by the Lexical Analyzer

5. DFA Design
 6. Implementation Methodology
 7. Program Code
 8. Sample Input
 9. Output Files
 10. Compilation and Execution
 11. Conclusion
-

1. Introduction

A **Lexical Analyzer** is the first phase of a compiler. It reads the source code **character by character** and converts the sequence of characters into meaningful tokens.

The lexical analyzer identifies:

- Keywords
- Identifiers
- Operators
- Delimiters
- Constants

These tokens are then passed to the **syntax analyzer** for further processing.

In this assignment, we design and implement a lexical analyzer in the **C programming language** using the **Ubuntu Linux environment**. The analyzer reads the input program from a file and generates a list of tokens along with a **symbol table**.

2. Objective

The objectives of this assignment are:

- To understand the working of a lexical analyzer.
 - To implement a lexical analyzer using the **C programming language**.
 - To read the input **character by character using an input buffer**.
 - To use **two pointers** to identify lexemes.
 - To simulate token recognition using **DFA logic**.
 - To generate a **token list and a symbol table**.
-

3. Background

Lexical analysis converts a sequence of characters into **tokens**. Each token represents a meaningful unit in programming languages.

Examples of tokens include:

Token Type	Example
------------	---------

Keyword	if, while
---------	-----------

Identifier	temp1
------------	-------

Operator	+, -, <
----------	---------

Delimiter	{ } ()
-----------	---------

Constant	3.5
----------	-----

The lexical analyzer scans the program and groups characters into **lexemes**, which are then classified as tokens.

4. Tokens Supported by the Lexical Analyzer

The lexical analyzer supports the following constructs:

Token	Description
+ - * /	Arithmetic operators
> < >= <= != ==	Relational operators
if	Conditional statement
while	Loop statement
break	Exit from a loop
print	Prints a value
{ }	Block delimiters
()	Expression delimiters

5. DFA Design

A **Deterministic Finite Automaton (DFA)** is used to recognize valid tokens.

Identifier DFA

Rules:

- Must start with a **letter**
- May contain **letters or digits afterward**

Example:

Letter → Letter/Digit → Identifier

Float DFA

Rules:

- Digits followed by a **decimal point**
- Followed by **digits**

Example:

Digit → Digit → . → Digit → Digit → FLOAT

DFA Transition Table for Lexical Analyzer

The lexical analyzer uses a **Deterministic Finite Automaton (DFA)** to recognize valid tokens such as identifiers and floating-point numbers. The DFA transitions between states depending on the input character type.

Character Classes

Symbol	Meaning
--------	---------

L	Letter (A–Z, a–z)
---	-------------------

D	Digit (0–9)
.	Decimal point
Other	Any other character

5.1 DFA for Identifier Recognition

Valid Identifier Rule

- Must start with a **letter**
- May contain **letters or digits afterward**

Transition Table

Current State	Input = Letter (L)	Input = Digit (D)	Other
q0 (start)	q1	Reject	Reject
q1	q1	q1	Accept

Explanation

- **q0 → q1**: Identifier begins with a letter
 - **q1 → q1**: Letters or digits continue the identifier
 - When another symbol appears, the lexeme is **accepted as an IDENTIFIER**
-

5.2 DFA for Floating Point Numbers

Valid Float Rule

- One or more digits
- Followed by a decimal point
- Followed by digits

Transition Table

Current State	Digit (D)	Decimal (.)	Other
q0 (start)	q1	Reject	Reject
q1	q1	q2	Accept Integer
q2	q3	Reject	Reject
q3	q3	Reject	Accept Float

Example

12.5

State Transitions

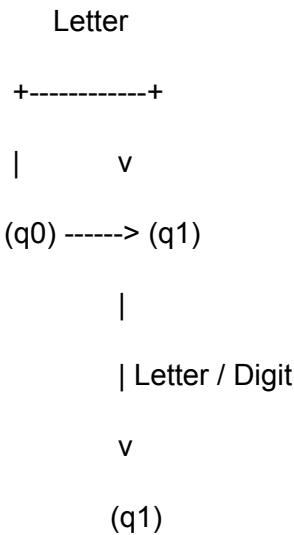
q0 → q1 → q1 → q2 → q3 → Accept

DFA for Operators and Delimiters

Operators and delimiters are recognized directly.

Character	Token Type
+ - * /	Arithmetic Operator
> < >= <= == !=	Relational Operator
{ }	Block Delimiter
()	Expression Delimiter

Simple DFA Diagram (Identifier Recognition)



Accepting State: **q1**

6. Implementation Methodology

The lexical analyzer works as follows:

1. The input program is **read character by character**.
 2. Characters are stored in an **input buffer**.
 3. **Two pointers** track the start and end of each lexeme.
 4. When a delimiter or operator is encountered, the lexeme is evaluated.
 5. A **DFA simulation function** checks the lexeme type.
 6. Valid tokens are written to the **token output file**.
 7. Identifiers are stored in the **symbol table**.
-

7. Program Code

The C program implementing the lexical analyzer is **attached separately** as [assignment1.c](#).

8. Sample Input (input.txt)

```
startFrom = 0
```

```
fib_last = 1
```

```
fib_secondLast = 1
```

```
while(startFrom < 7)
```

```
{
```

```
fib_next = fib_last + fib_secondLast
```

```
fib_secondLast = fib_last
```

```
fib_last = fib_next
```

```
startFrom = startFrom + 1  
}
```

```
print(fib_last)
```

9. Output Files

tokens.txt

```
<IDENTIFIER, startFrom>
```

```
<OPERATOR, ==>
```

```
<FLOAT, 0>
```

```
<IDENTIFIER, fib_last>
```

```
<OPERATOR, ==>
```

```
<FLOAT, 1>
```

```
<KEYWORD, while>
```

```
<DELIMITER, (>
```

```
...
```

symbol_table.txt

Symbol Table

1 startFrom

2 fib_last

3 fib_secondLast

4 fib_next

10. Compilation and Execution

Compile the program

```
gcc assignment1.c -o Assign1
```

Run the program

```
./Assign1
```

Generated Files

tokens.txt

symbol_table.txt

11. Conclusion

In this assignment, a **lexical analyzer** was successfully implemented in the **C programming language**. The analyzer reads source code character by character and identifies tokens such as keywords, identifiers, operators, and delimiters.

The program also generates a **symbol table containing identifiers** found in the input program. This project demonstrates the fundamental concept of **lexical analysis and its role as the first stage of compiler design**.

```
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ gcc LAB3C451.c -o LAB#3
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: return
Invalid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: my_var1
Valid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: a123
Valid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: _temp9
Valid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: 1stVar
Invalid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: for
Invalid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: return
Invalid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input:
Valid Identifier
admin-1@admin-1-OptiPlex-5400-AIO:~/Desktop$ ./LAB#3
Enter input: x9
```